

Dan Boneh 密码学专题：哈希函数的定义与作用

本笔记用于快速复习密码学哈希函数的基本定义、安全性质、主要作用和常见误区。

1. 一句话定义

哈希函数把任意长度或较长的输入消息，映射为一个固定长度的短字符串：

$$H : \mathcal{M} \rightarrow \mathcal{T}, \quad |\mathcal{M}| \gg |\mathcal{T}|.$$

其中：

- $m \in \mathcal{M}$ 是输入消息；
- $H(m) \in \mathcal{T}$ 是哈希值；
- 哈希值也叫摘要、消息摘要或指纹；
- 若输出长度为 n 位，则 $|\mathcal{T}| = 2^n$ 。

例如 SHA-256 不论输入是一句话、一张图片还是一个很大的文件，输出始终为 256 位，即 32 字节。

2. 哈希函数的基本特征

2.1 确定性

同一条消息每次都产生相同哈希值：

$$m = m' \implies H(m) = H(m').$$

哈希函数通常不使用随机数。

2.2 输入长度灵活

密码学哈希函数通常可以处理任意长度或非常长的输入：

```
H("a")  
H("一篇文章")  
H("一个 10GB 文件")
```

三者最终都产生固定长度的摘要。

2.3 输出长度固定

以 SHA-256 为例：

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}.$$

$\{0, 1\}^*$ 表示任意有限长度的比特串，输出始终是 256 位。

2.4 计算高效

给定消息 m ，应当能够快速计算 $H(m)$ 。哈希函数常被用于大文件，因此吞吐量通常很重要。

2.5 公开且通常不带密钥

普通密码学哈希算法是公开的：

- 算法结构公开；
- 任何人都可以计算；
- 通常没有秘密密钥。

因此，哈希函数本身不负责证明“是谁计算或发送了这个摘要”。

3. 压缩意味着碰撞必然存在

哈希函数把巨大输入空间压缩到有限输出空间。根据抽屉原理，必然存在两条不同消息满足：

$$m_0 \neq m_1, \quad H(m_0) = H(m_1).$$

这称为**哈希碰撞**。

因此，不能要求密码学哈希函数“绝对没有碰撞”。正确要求是：

碰撞虽然一定存在，但现实中的高效攻击者应当难以找到。

4. 密码学哈希函数的三项核心安全性质

数学意义上的哈希函数只要求完成映射。密码学哈希函数还需要满足额外的计算安全性质。

4.1 原像抗性

给定一个目标哈希值 y ，攻击者难以找到消息 m ，使：

$$H(m) = y.$$

可以记成：

已知摘要 y
↓
难以反推出任意一个对应消息 m

对于理想的 n 位哈希函数，通用原像攻击约需要：

$$O(2^n)$$

次尝试。

原像抗性常被通俗地称为“单向性”，但它不是说数学上绝对不可逆，而是说计算上难以寻找原像。

4.2 第二原像抗性

给定一条指定消息 m_0 ，攻击者难以找到另一条不同消息 m_1 ，使：

$$m_1 \neq m_0, \quad H(m_1) = H(m_0).$$

对于理想的 n 位哈希函数，通用第二原像攻击通常约需要：

$$O(2^n)$$

次尝试。

它适合描述这种攻击：攻击者已经看到一份确定的合法文件，希望制作另一份具有相同摘要的文件。

4.3 抗碰撞性

攻击者可以自己选择两条消息，但仍难以找到：

$$m_0 \neq m_1, \quad H(m_0) = H(m_1).$$

与第二原像攻击不同，碰撞攻击中两条消息都由攻击者自由选择。

对于理想的 n 位哈希函数，生日攻击只需大约：

$$O(2^{n/2})$$

次计算。因此，256 位理想哈希函数只能提供大约 128 位的通用抗碰撞安全性。

4.4 三者对照

性质	攻击者已知什么	攻击目标	理想 n 位哈希的通用复杂度
原像抗性	指定摘要 y	找 m ，使 $H(m) = y$	约 2^n
第二原像抗性	指定消息 m_0	找不同的 m_1 ，摘要与 m_0 相同	约 2^n
抗碰撞性	没有指定目标	自由寻找任意一对碰撞消息	约 $2^{n/2}$

这三种性质的攻击条件不同，不能简单地当成同一个概念。

5. 哈希函数的主要作用

5.1 生成数据指纹

哈希值可以作为大数据的短指纹：

$$\text{大文件} \xrightarrow{H} \text{固定长度摘要}.$$

比较两个短摘要通常比逐字节比较两个大文件更方便。

如果：

$$H(m) \neq H(m'),$$

则一定有：

$$m \neq m'.$$

反过来，如果摘要相同，不能从数学上断言消息绝对相同，因为碰撞存在；对于安全哈希，可以说攻击者很难刻意制造这种情况。

5.2 文件完整性校验

从不可信网络下载文件后，可以计算：

$$H(\text{downloaded file})$$

并与从可信渠道获得的摘要 h 比较：

$$H(\text{downloaded file}) \stackrel{?}{=} h.$$

如果不同，文件一定发生了变化。

但安全性的关键是：**正确摘要必须来自可信位置**。如果攻击者可以同时替换文件和网页上的摘要，它可以重新计算恶意文件的哈希，普通哈希无法发现问题。

5.3 数字签名中的消息压缩

公钥签名算法通常不直接签署任意长文件，而是先计算摘要：

$$h = H(m),$$

再对经过规范编码的摘要进行签名：

$$\sigma = \text{Sign}(sk, h).$$

这样做可以：

- 把任意长消息压缩成固定长度；
- 提高签名效率；
- 让签名算法只处理规定长度的输入。

如果哈希函数不抗碰撞，攻击者可能构造合法消息 m_0 和恶意消息 m_1 ，使：

$$H(m_0) = H(m_1),$$

从而把 m_0 的签名转移到 m_1 上。

实际签名必须使用经过标准化设计的完整签名方案，不能简单地自行实现“裸哈希后签名”。

5.4 消息认证 HMAC

普通哈希函数不带密钥，不能单独证明消息来源。若需要消息完整性和来源认证，可以使用 HMAC：

$$\text{HMAC}(k, m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m)).$$

只有知道秘密密钥 k 的一方才能生成正确标签。

应当区分：

工具	是否带密钥	能否认证来源
普通哈希 $H(m)$	否	不能
HMAC\$(k,m)\$	是	能，在密钥安全和方案正确使用的条件下

5.5 密钥派生

哈希函数和 HMAC 可以作为密钥派生函数的组成部分。例如 HKDF 使用 HMAC 完成 Extract-then-Expand:

$$PRK = HMAC(salt, IKM).$$

然后结合上下文和计数器生成多段密钥材料。

这里应使用标准 KDF，而不是随意计算 $H(secret)$ 或拼接字符串，因为标准 KDF 会明确处理 salt、上下文、输出长度和域分离。

5.6 内容寻址和去重

系统可以用内容摘要标识对象：

$$id = H(content).$$

相同内容会得到相同标识，可用于：

- 版本控制对象；
- 内容寻址存储；
- 数据去重；
- 缓存键；
- 大文件分块校验。

如果应用面对恶意输入，应选用仍然安全的密码学哈希算法，并考虑碰撞与输入编码问题。

5.7 数据结构完整性

Merkle Tree 把大量数据块的哈希逐层组合，最终得到一个短的根哈希。只需一条较短的认证路径，就能验证某个数据块是否属于被根哈希承诺的数据集合。

典型应用包括版本控制、分布式存储和区块链数据结构。

6. 哈希函数不能做什么

6.1 哈希不是加密

加密的目标是让拥有密钥的人可以恢复明文：

$$D(k, E(k, m)) = m.$$

哈希的目标是压缩数据，没有对应的解密密钥或标准逆算法。

哈希	加密
通常不带密钥	必须带密钥
输出长度固定	密文长度通常随明文增长
不要求可逆	必须可逆
用于摘要和完整性结构	用于保密性

6.2 普通哈希不能提供保密性

如果消息空间很小，攻击者可以枚举所有候选并比较摘要。例如对四位 PIN：

1. 枚举 0000 到 9999；
2. 计算每个候选的哈希；
3. 与目标哈希比较。

即使哈希函数具有原像抗性，小消息空间仍然可以被穷举。

6.3 普通哈希不能认证发送者

任何人都能计算 $H(m)$ 。攻击者修改消息后，也能计算：

$$H(m').$$

因此，单独发送：

$$m \parallel H(m)$$

不能抵抗主动篡改。需要 HMAC、数字签名或认证加密。

6.4 哈希不能保证输入唯一

因为碰撞必然存在，所以不能把哈希值当作没有任何冲突可能的数学唯一编号。

在数据库中使用哈希作为索引时，仍应正确处理摘要冲突；在安全协议中则依赖抗碰撞性让恶意冲突难以构造。

6.5 快速哈希不适合直接保存密码

SHA-256 设计得很快，但密码存储希望攻击者的每次猜测都很昂贵。

因此，不应直接保存：

$$SHA256(password).$$

应使用带独立随机 salt 的专用密码哈希/KDF，例如：

- Argon2id；
- scrypt；
- bcrypt；
- 在兼容场景中使用 PBKDF2。

这些算法通过时间成本和内存成本减慢离线字典攻击。

7. 哈希、校验和、MAC 与签名的区别

工具	是否带秘密	主要防御对象	能提供什么	不能提供什么
CRC/校验和	否	随机传输错误	快速检测偶然错误	不能抵抗恶意篡改
密码学哈希	否	依赖可信摘要的篡改检测	数据指纹、抗碰撞摘要	单独不能认证来源
HMAC	是，共享密钥	主动篡改与伪造	完整性和对称认证	不提供公开可验证性
数字签名	是，私钥签名	主动篡改与身份冒充	完整性、来源认证、公开验证	不自动提供保密性
认证加密	是，共享密钥	窃听和主动篡改	保密性、完整性和认证	不自动防重放和长度泄露

8. 常见哈希算法

8.1 现代常用算法

- SHA-256;
- SHA-384;
- SHA-512;
- SHA-3 系列;
- BLAKE2、BLAKE3，在适当应用中使用。

8.2 不应继续用于抗碰撞安全的新设计

- MD5 已存在实用碰撞攻击;
- SHA-1 已存在实用碰撞攻击。

它们可能仍出现在非安全校验或遗留系统中，但不应在新签名、证书和依赖抗碰撞性的安全设计中使用。

算法选择还应结合协议标准和密码库接口，不要只按输出长度自行判断。

9. 一个完整例子：验证软件下载

假设用户从镜像站下载程序 *file*。

情况一：摘要来自同一个可被篡改的镜像站

镜像站同时提供：

```
program.exe  
SHA-256(program.exe)
```

攻击者若控制镜像站，可以替换程序并重新计算摘要。此时哈希比较仍会通过，不能证明文件可信。

情况二：摘要通过可信签名发布

软件发布者对文件摘要签名：

$$h = H(file),$$
$$\sigma = \text{Sign}(sk, h).$$

用户用发布者的可信公钥验证签名，再比较下载文件的摘要。攻击者无法在不知道私钥的情况下为恶意文件生成有效签名。

因此，哈希负责把大文件压缩成指纹，签名负责证明指纹来自发布者。二者承担不同任务。

10. 常见误区

误区 1：哈希函数绝对不会碰撞

错误。输入空间远大于输出空间，碰撞必然存在。抗碰撞性要求的是高效攻击者难以找到。

误区 2：两个文件哈希相同，所以数学上绝对相同

错误。只能说在安全哈希假设下，恶意制造不同文件却摘要相同在计算上不可行。

误区 3：哈希值不可读，所以哈希提供加密

错误。哈希不是可逆加密，也不能阻止小空间枚举。

误区 4：给消息附加哈希就能防篡改

错误。攻击者可以同时修改消息和哈希。必须通过可信渠道获得摘要，或使用 HMAC、签名、认证加密。

误区 5：原像抗性和抗碰撞性相同

错误。原像攻击给定摘要，碰撞攻击可以自由选择两条消息；二者的通用复杂度分别约为 2^n 和 $2^{n/2}$ 。

误区 6：256 位哈希具有 256 位抗碰撞安全性

错误。生日攻击使理想 256 位哈希的通用抗碰撞安全性约为 128 位。

误区 7：SHA-256 可以直接安全保存密码

错误。SHA-256 太快，不能有效减慢离线猜测。密码存储需要专用慢速、带 salt 的密码哈希算法。

误区 8：摘要需要像密钥一样保密

通常错误。摘要一般可以公开，关键是保证正确摘要的来源可信且未被替换。

11. 必记公式

哈希函数：

$$H : \mathcal{M} \rightarrow \mathcal{T}, \quad |\mathcal{M}| \gg |\mathcal{T}|.$$

n 位哈希：

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

碰撞：

$$m_0 \neq m_1, \quad H(m_0) = H(m_1).$$

原像目标：

$$\text{给定 } y, \text{ 寻找 } m \text{ 使 } H(m) = y.$$

第二原像目标：

$$\text{给定 } m_0, \text{ 寻找 } m_1 \neq m_0 \text{ 使 } H(m_1) = H(m_0).$$

理想哈希的通用复杂度：

$$\text{原像和第二原像：约 } 2^n,$$

$$\text{碰撞：约 } 2^{n/2}.$$

12. 自测题

题目

1. 密码学哈希函数的输入和输出分别有什么特点？
2. 为什么碰撞一定存在？
3. “抗碰撞”是否表示哈希函数没有碰撞？
4. 原像抗性、第二原像抗性和抗碰撞性的攻击目标分别是什么？
5. 为什么 n 位哈希的抗碰撞复杂度约为 $2^{n/2}$ ？
6. 哈希怎样用于验证下载文件完整性？其中最重要的信任条件是什么？
7. 为什么 $m \parallel H(m)$ 不能抵抗主动篡改？
8. 哈希函数和 HMAC 的核心区别是什么？
9. 数字签名为什么通常先对长消息计算哈希？
10. 哈希函数为什么不是加密算法？
11. 为什么不能直接使用 SHA-256 保存用户密码？
12. MD5 和 SHA-1 为什么不适合新的抗碰撞安全设计？

参考答案

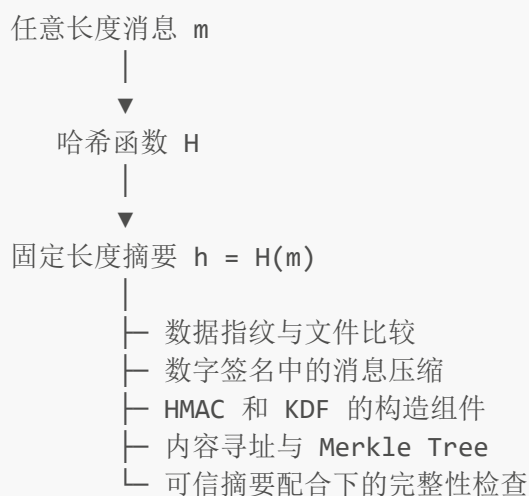
1. 输入可以是任意长度或很长的消息，输出是固定长度摘要；算法确定、公开且计算高效。
2. 输入空间远大于有限输出空间，根据抽屉原理必然有不同输入映射到相同输出。

3. 不是。它表示现实中的高效攻击者难以找到任意一对碰撞。
4. 原像攻击给定摘要找消息；第二原像攻击给定消息找另一条同摘要消息；碰撞攻击自由寻找任意一对同摘要消息。
5. 约 q 个摘要形成约 $q^2/2$ 对；当 q^2 与 2^n 同量级时，碰撞概率已经达到常数量级。
6. 计算下载文件摘要并与可信摘要比较。正确摘要必须来自攻击者无法同时替换的可信渠道。
7. 哈希不带密钥，攻击者修改消息后可以自行计算新消息的哈希并一起替换。
8. 普通哈希不带密钥，任何人都能计算；HMAC 使用共享秘密密钥，可以提供消息完整性和来源认证。
9. 摘要把任意长消息压缩为固定长度，提高签名效率，并让签名算法处理规范长度输入。
10. 哈希不要求可逆，没有解密密钥；加密必须让持有密钥者恢复原文。
11. SHA-256 速度太快，攻击者可以高速枚举口令；应使用带随机 salt 和较高成本的专用密码哈希/KDF。
12. 二者已经存在实用碰撞攻击，不能再提供设计所需的抗碰撞保证。

13. 一分钟复习

1. 哈希函数把任意长消息映射为固定长度摘要。
2. 哈希通常公开、确定、高效且不带密钥。
3. 压缩必然产生碰撞；密码学要求碰撞难以找到，而不是不存在。
4. 三项核心性质是原像抗性、第二原像抗性和抗碰撞性。
5. 理想 n 位哈希的原像复杂度约 2^n ，碰撞复杂度约 $2^{n/2}$ 。
6. 哈希可用于数据指纹、文件校验、数字签名预处理、HMAC/KDF 和内容寻址。
7. 文件校验只有在正确摘要来自可信渠道时才能抵抗恶意替换。
8. 普通哈希不能提供保密性，也不能单独认证发送者。
9. HMAC 是带密钥的消息认证方案，不等同于普通哈希。
10. 哈希不是加密，摘要也不是数学上绝对唯一的编号。
11. 快速哈希不能直接用于密码存储，应使用 Argon2id、scrypt 等专用方案。
12. 新的抗碰撞安全设计不应继续使用 MD5 或 SHA-1。

14. 最简关系图



但普通哈希不能单独提供：

- ├ 保密性
- └ 发送者认证

- └ 防止主动替换消息和摘要
- └ 密码存储所需的慢速猜测成本